

PES-VCS Final Report

Name	Vageesh Hiremath
SRN	PES2UG24CS663
GitHub Repository	https://github.com/Vageeshhiremath/PES2UG24CS663-pes-vcs

This project implements a simplified Git-like Version Control System using content-addressable storage, tree structures, and commit history.

Phase 1: Object Storage Foundation

In this phase, I implemented content-addressable object storage using SHA-256 hashing.

Objects are stored inside .pes/objects using sharded directories, and object integrity is verified during reads.

Modified Code: object_write and object_read

```
int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out) {
    const char *type_str;
    if (!data || !id_out) return -1;

    switch (type) {
        case OBJ_BLOB: type_str = "blob"; break;
        case OBJ_TREE: type_str = "tree"; break;
        case OBJ_COMMIT: type_str = "commit"; break;
        default: return -1;
    }

    char header[64];
    int header_len = snprintf(header, sizeof(header), "%s %zu", type_str, len);
    if (header_len < 0 || (size_t)header_len + 1 > sizeof(header)) return -1;

    size_t full_len = (size_t)header_len + 1 + len;
    unsigned char *full_object = malloc(full_len);
    if (!full_object) return -1;

    memcpy(full_object, header, (size_t)header_len);
    full_object[header_len] = '\0';
    memcpy(full_object + header_len + 1, data, len);

    ObjectID id;
    compute_hash(full_object, full_len, &id);
    *id_out = id;

    if (object_exists(&id)) { free(full_object); return 0; }

    char final_path[512];
    object_path(&id, final_path, sizeof(final_path));

    char shard_dir[512];
    snprintf(shard_dir, sizeof(shard_dir), "%s", final_path);
    char *slash = strchr(shard_dir, '/');
    if (!slash) { free(full_object); return -1; }
    *slash = '\0';

    if (mkdir(shard_dir, 0755) != 0 && access(shard_dir, F_OK) != 0) {
        free(full_object); return -1;
    }

    char temp_path[512];
    snprintf(temp_path, sizeof(temp_path), "%s.tmp.%ld", final_path, (long)getpid());
    int fd = open(temp_path, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { free(full_object); return -1; }

    size_t written = 0;
    while (written < full_len) {
        ssize_t n = write(fd, full_object + written, full_len - written);
        if (n <= 0) { close(fd); unlink(temp_path); free(full_object); return -1; }
        written += (size_t)n;
    }

    if (fsync(fd) != 0) { close(fd); unlink(temp_path); free(full_object); return -1; }
    if (close(fd) != 0) { unlink(temp_path); free(full_object); return -1; }
    if (rename(temp_path, final_path) != 0) { unlink(temp_path); free(full_object); return -1; }

    int dir_fd = open(shard_dir, O_RDONLY);
    if (dir_fd >= 0) { fsync(dir_fd); close(dir_fd); }
    free(full_object);
    return 0;
}

int object_read(const ObjectID *id, ObjectType *type_out, void **data_out, size_t *len_out) {
    if (!id || !type_out || !data_out || !len_out) return -1;

    char path[512];
    object_path(id, path, sizeof(path));
    FILE *f = fopen(path, "rb");
    if (!f) return -1;

    if (fseek(f, 0, SEEK_END) != 0) { fclose(f); return -1; }
    long file_size = ftell(f);
    if (file_size < 0) { fclose(f); return -1; }
    rewind(f);

    unsigned char *buffer = malloc((size_t)file_size);
    if (!buffer) { fclose(f); return -1; }
    size_t bytes_read = fread(buffer, 1, (size_t)file_size, f);
    fclose(f);
```

```

if (bytes_read != (size_t)file_size) { free(buffer); return -1; }

ObjectID actual_id;
compute_hash(buffer, (size_t)file_size, &actual_id);
if (memcmp(actual_id.hash, id->hash, HASH_SIZE) != 0) { free(buffer); return -1; }

unsigned char *nul = memchr(buffer, '\0', (size_t)file_size);
if (!nul) { free(buffer); return -1; }

size_t header_len = (size_t)(nul - buffer);
char header[128];
if (header_len >= sizeof(header)) { free(buffer); return -1; }
memcpy(header, buffer, header_len);
header[header_len] = '\0';

char type_str[16];
size_t data_len;
if (sscanf(header, "%15s %zu", type_str, &data_len) != 2) { free(buffer); return -1; }

if (strcmp(type_str, "blob") == 0) *type_out = OBJ_BLOB;
else if (strcmp(type_str, "tree") == 0) *type_out = OBJ_TREE;
else if (strcmp(type_str, "commit") == 0) *type_out = OBJ_COMMIT;
else { free(buffer); return -1; }

size_t actual_data_len = (size_t)file_size - header_len - 1;
if (actual_data_len != data_len) { free(buffer); return -1; }

void *data = malloc(data_len);
if (!data && data_len > 0) { free(buffer); return -1; }
memcpy(data, nul + 1, data_len);
*data_out = data;
*len_out = data_len;
free(buffer);
return 0;
}

```

Phase 1: Object Storage Foundation - Output Screenshots

Screenshot 1A: ./test_objects Output

```
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc1
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5
b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: .pes/objects Directory Structure

```
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$
```

Phase 2: Tree Objects

In this phase, I implemented tree construction from the index to represent directory snapshots.

Tree objects store file and directory entries using modes, names, and object hashes.

Modified Code: `tree_from_index` and helpers

```
static int tree_has_entry(const Tree *tree, const char *name) {
    for (int i = 0; i < tree->count; i++) {
        if (strcmp(tree->entries[i].name, name) == 0) return 1;
    }
    return 0;
}

static int write_tree_level(const Index *index, const char *prefix, ObjectID *id_out) {
    Tree tree;
    tree.count = 0;
    size_t prefix_len = strlen(prefix);

    for (int i = 0; i < index->count; i++) {
        const char *path = index->entries[i].path;
        if (strncmp(path, prefix, prefix_len) != 0) continue;

        const char *remaining = path + prefix_len;
        if (remaining[0] == '\\0') continue;

        const char *slash = strchr(remaining, '/');
        if (slash == NULL) {
            if (tree.count >= MAX_TREE_ENTRIES) return -1;
            TreeEntry *entry = &tree.entries[tree.count++];
            entry->mode = index->entries[i].mode;
            entry->hash = index->entries[i].hash;
            snprintf(entry->name, sizeof(entry->name), "%s", remaining);
        } else {
            size_t dir_len = (size_t)(slash - remaining);
            char dir_name[256];
            if (dir_len == 0 || dir_len >= sizeof(dir_name)) return -1;
            memcpy(dir_name, remaining, dir_len);
            dir_name[dir_len] = '\\0';

            if (tree_has_entry(&tree, dir_name)) continue;
            if (tree.count >= MAX_TREE_ENTRIES) return -1;

            char child_prefix[512];
            int written = snprintf(child_prefix, sizeof(child_prefix), "%s%s/", prefix, dir_name);
            if (written < 0 || (size_t)written >= sizeof(child_prefix)) return -1;

            ObjectID child_id;
            if (write_tree_level(index, child_prefix, &child_id) != 0) return -1;

            TreeEntry *entry = &tree.entries[tree.count++];
            entry->mode = MODE_DIR;
            entry->hash = child_id;
            snprintf(entry->name, sizeof(entry->name), "%s", dir_name);
        }
    }

    void *data = NULL;
    size_t len = 0;
    if (tree_serialize(&tree, &data, &len) != 0) return -1;
    int rc = object_write(OBJ_TREE, data, len, id_out);
    free(data);
    return rc;
}

int tree_from_index(ObjectID *id_out) {
    if (!id_out) return -1;
    if (index_load == NULL) return -1;

    Index index;
    if (index_load(&index) != 0) return -1;
    return write_tree_level(&index, "", id_out);
}
```

Phase 2: Tree Objects - Output Screenshots

Screenshot 2A: ./test_tree Output

```
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Raw Object Using xxd

```
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ find .pes/objects -type f
.pes/objects/b4/e76aee38854839557e5a3132e346adbf8288ec2ba878f2cd915d5c2326af00
.pes/objects/b7/d498243902dc5aafed14e9179bf9267fd925a8705d7e88e70d5dbc87778c2d
.pes/objects/f6/8aeaab18272fa2c862dc8721e1ddfbaba520a9b87953adeeb24a7d9412407f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/8c/c96f3ca2e3ad1e870bc95ec2f6da779fa0ab98a1c5af18ea1a3073c9eaf55a
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f7/645d85bf85991d2b3f20feba1a36088b7c6b5093d28b7b43e431491c706c7d
.pes/objects/e4/329ff9662ebec3c7589983c9c43219d408b4de5c3cb6305ddca11ffd1b102d
.pes/objects/c7/187e8fdb691b3a692e5f3f0bbcb6359e5046285225f18f9773d4fe54268c55
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ xxd .pes/objects/b4/e76aee38854839
7e5a3132e346adbf8288ec2ba878f2cd915d5c2326af00 | head -20
00000000: 626c 6f62 2031 3200 6865 6c6c 6f0a 576f  blob 12.hello.Wo
00000010: 726c 640a                                rld.
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$
```

Phase 3: Index / Staging Area

In this phase, I implemented index loading, saving, and file staging.

The index stores staged file metadata including mode, blob hash, modification time, size, and path.

Modified Code: index_load, index_save and index_add

```
int index_load(Index *index) {
    if (!index) return -1;
    index->count = 0;

    FILE *f = fopen(INDEX_FILE, "r");
    if (!f) return 0;

    while (index->count < MAX_INDEX_ENTRIES) {
        IndexEntry *entry = &index->entries[index->count];
        char hash_hex[HASH_HEX_SIZE + 1];
        unsigned int mode;
        unsigned long long mtime;
        unsigned int size;
        char path[sizeof(entry->path)];

        int rc = fscanf(f, "%o %64s %llu %u %511s", &mode, hash_hex, &mtime, &size, path);
        if (rc == EOF) break;
        if (rc != 5) { fclose(f); return -1; }
        if (hex_to_hash(hash_hex, &entry->hash) != 0) { fclose(f); return -1; }

        entry->mode = mode;
        entry->mtime_sec = (uint64_t)mtime;
        entry->size = size;
        snprintf(entry->path, sizeof(entry->path), "%s", path);
        index->count++;
    }
    fclose(f);
    return 0;
}

static int compare_index_entries(const void *a, const void *b) {
    const IndexEntry *entry_a = (const IndexEntry *)a;
    const IndexEntry *entry_b = (const IndexEntry *)b;
    return strcmp(entry_a->path, entry_b->path);
}

int index_save(const Index *index) {
    if (!index) return -1;

    IndexEntry *sorted = malloc((size_t)index->count * sizeof(IndexEntry));
    if (!sorted && index->count > 0) return -1;
    memcpy(sorted, index->entries, (size_t)index->count * sizeof(IndexEntry));
    qsort(sorted, index->count, sizeof(IndexEntry), compare_index_entries);

    FILE *f = fopen(INDEX_FILE ".tmp", "w");
    if (!f) { free(sorted); return -1; }

    for (int i = 0; i < index->count; i++) {
        char hash_hex[HASH_HEX_SIZE + 1];
        hash_to_hex(&sorted[i].hash, hash_hex);
        if (fprintf(f, "%o %s %llu %u %s\n", sorted[i].mode, hash_hex,
                    (unsigned long long)sorted[i].mtime_sec,
                    sorted[i].size, sorted[i].path) < 0) {
            fclose(f); free(sorted); return -1;
        }
    }
    free(sorted);

    if (fflush(f) != 0) { fclose(f); return -1; }
    if (fsync(fileno(f)) != 0) { fclose(f); return -1; }
    if (fclose(f) != 0) return -1;
    if (rename(INDEX_FILE ".tmp", INDEX_FILE) != 0) return -1;
    return 0;
}

int index_add(Index *index, const char *path) {
    if (!index || !path) return -1;

    struct stat st;
    if (stat(path, &st) != 0) return -1;
    if (!S_ISREG(st.st_mode)) return -1;

    FILE *f = fopen(path, "rb");
    if (!f) return -1;

    unsigned char *buffer = malloc((size_t)st.st_size);
    if (!buffer && st.st_size > 0) { fclose(f); return -1; }
    size_t bytes_read = fread(buffer, 1, (size_t)st.st_size, f);
    fclose(f);
    if (bytes_read != (size_t)st.st_size) { free(buffer); return -1; }
    ObjectID blob_id;
```

```

if (object_write(OBJ_BLOB, buffer, (size_t)st.st_size, &blob_id) != 0) {
    free(buffer); return -1;
}
free(buffer);

IndexEntry *entry = index_find(index, path);
if (!entry) {
    if (index->count >= MAX_INDEX_ENTRIES) return -1;
    entry = &index->entries[index->count++];
}

uint32_t mode = get_file_mode(path);
if (mode == 0) return -1;

entry->mode = mode;
entry->hash = blob_id;
entry->mtime_sec = (uint64_t)st.st_mtime;
entry->size = (uint32_t)st.st_size;
snprintf(entry->path, sizeof(entry->path), "%s", path);
return index_save(index);
}

```


Phase 3: Index / Staging Area - Output Screenshots

Screenshot 3A: pes init, pes add, and pes status

```
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ echo "hello" > file1.txt
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ echo "world" > file2.txt
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ ./pes add file1.txt file2.txt
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ ./pes status
Staged changes:
  staged:    file1.txt
  staged:    file2.txt

Unstaged changes:
(nothing to show)

Untracked files:
  untracked: test_tree.c
  untracked: commit.h
  untracked: test_objects
  untracked: index.c
  untracked: commit.c
  untracked: tree.h
  untracked: tree.c
  untracked: index.h
  untracked: test_tree
  untracked: Makefile
  untracked: README.md
  untracked: test_sequence.sh
  untracked: .gitignore
  untracked: pes.c
  untracked: pes.h
  untracked: object.c
  untracked: test_objects.c
```

Screenshot 3B: .pes/index Contents

```
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776777260 6 fil
e1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776777285 6 fil
e2.txt
```

Phase 4: Commits and History

In this phase, I implemented commit creation using the staged index and generated root tree object.

Each commit stores tree hash, parent commit, author, timestamp, and commit message.

Modified Code: commit_create

```
int commit_create(const char *message, ObjectID *commit_id_out) {
    if (!message || !commit_id_out) return -1;

    Commit commit;
    memset(&commit, 0, sizeof(commit));

    if (tree_from_index(&commit.tree) != 0) return -1;

    if (head_read(&commit.parent) == 0) commit.has_parent = 1;
    else commit.has_parent = 0;

    snprintf(commit.author, sizeof(commit.author), "%s", pes_author());
    commit.timestamp = (uint64_t)time(NULL);
    snprintf(commit.message, sizeof(commit.message), "%s", message);

    void *data = NULL;
    size_t len = 0;
    if (commit_serialize(&commit, &data, &len) != 0) return -1;

    if (object_write(OBJ_COMMIT, data, len, commit_id_out) != 0) {
        free(data);
        return -1;
    }
    free(data);

    if (head_update(commit_id_out) != 0) return -1;
    return 0;
}
```

Phase 4: Commits and History - Output Screenshots

Screenshot 4A: ./pes log Output

```
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ ./pes log
commit b7d498243902dc5aafed14e9179bf9267fd925a8705d7e88e70d5dbc87778c2d
Author: PES User <pes@localhost>
Date: 1776778309

    Add farewell

commit 8cc96f3ca2e3ad1e870bc95ec2f6da779fa0ab98a1c5af18ea1a3073c9eaf55a
Author: PES User <pes@localhost>
Date: 1776778287

    Add worldell

commit f7645d85bf85991d2b3f20feba1a36088b7c6b5093d28b7b43e431491c706c7d
Author: PES User <pes@localhost>
Date: 1776778184

    Initial committial commiQ

vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$
```

Screenshot 4B: ./pes Files After Three Commits

```
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/8c/c96f3ca2e3ad1e870bc95ec2f6da779fa0ab98a1c5af18ea1a3073c9eaf55a
.pes/objects/b4/e76aee38854839557e5a3132e346adbfb8288ec2ba878f2cd915d5c2326af00
.pes/objects/b7/d498243902dc5aafed14e9179bf9267fd925a8705d7e88e70d5dbc87778c2d
.pes/objects/c7/187e8fdb691b3a692e5f3f0bbcb6359e5046285225f18f9773d4fe54268c55
.pes/objects/e4/329ff9662ebec3c7589983c9c43219d408b4de5c3cb6305ddca11ffd1b102d
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f6/8aeaab18272fa2c862dc8721e1ddfbaba520a9b87953adeeb24a7d9412407f
.pes/objects/f7/645d85bf85991d2b3f20feba1a36088b7c6b5093d28b7b43e431491c706c7d
.pes/refs/heads/main
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$
```

Screenshot 4C: HEAD and Branch Reference

```
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ cat .pes/refs/heads/main
b7d498243902dc5aafed14e9179bf9267fd925a8705d7e88e70d5dbc87778c2d
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$
```

Final Integration Test

The final integration test verifies repository initialization, staging, committing, logging, reference updates, and object store creation.

Final Integration Test - Part 1

```
vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:    file.txt
  staged:    hello.txt

Unstaged changes:
(nothing to show)

Untracked files:
(nothing to show)

--- First Commit ---
Committed: a25ee47206f8... Initial commit

Log after first commit:
commit a25ee47206f800f47b5aed649a2825c2efabe0e9c6d6c31c7fcd756ac32ccdbf
Author: PES User <pes@localhost>
Date:   1776778550

    Initial commit

--- Second Commit ---
Committed: 8d629d2214b0... Update file.txt

--- Third Commit ---
Committed: fafe9715bd93... Add farewell

--- Full History ---
commit fafe9715bd93e749a5cd772ba9e0e0b830fb7e2196459279eb8130727ffcd0d8
```

Final Integration Test - Part 2

Add farewell

```
commit 8d629d2214b084a274cddb0c3acd117861099068c65c884f1a1af1f5eab00b58
Author: PES User <pes@localhost>
Date: 1776778550
```

Update file.txt

```
commit a25ee47206f800f47b5aed649a2825c2efabe0e9c6d6c31c7fcd756ac32ccdbf
Author: PES User <pes@localhost>
Date: 1776778550
```

Initial committial commiQ

--- Reference Chain ---

HEAD:

ref: refs/heads/main

refs/heads/main:

fafe9715bd93e749a5cd772ba9e0e0b830fb7e2196459279eb8130727ffcd0d8

--- Object Store ---

Objects created:

10

```
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/8d/629d2214b084a274cddb0c3acd117861099068c65c884f1a1af1f5eab00b58
.pes/objects/a2/5ee47206f800f47b5aed649a2825c2efabe0e9c6d6c31c7fcd756ac32ccdbf
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafaaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/fa/fe9715bd93e749a5cd772ba9e0e0b830fb7e2196459279eb8130727ffcd0d8
```

=== All integration tests completed ===

vageesh@vageesh-VirtualBox:~/PES2UG24CS663-pes-vcs\$

Analysis Questions

Q5.1: Implementing pes checkout

pes checkout would first check whether `.pes/refs/heads/` exists. If it exists, `.pes/HEAD` would be updated to contain `ref: refs/heads/`. After updating HEAD, PES-VCS would read the commit hash from that branch, load the commit object, load its root tree, and update the working directory to match that tree snapshot.

Q5.2: Dirty Working Directory Detection

A dirty working directory conflict can be detected by comparing tracked files in the working directory with the index. PES-VCS can check file size, modification time, and, if needed, rehash file contents and compare them with the stored blob hash. If a tracked file has uncommitted changes and the target branch contains a different version of that file, checkout should refuse to continue.

Q5.3: Detached HEAD

Detached HEAD means `.pes/HEAD` stores a commit hash directly instead of pointing to a branch reference. New commits can still be created, but no branch name automatically points to them. The user can recover those commits by creating a new branch file inside `.pes/refs/heads/` containing the detached commit hash.

Q6.1: Garbage Collection

Garbage collection would start from all branch references in `.pes/refs/heads/` and mark every reachable commit. From each commit, it would recursively visit its tree objects, subtrees, and blob objects. All reachable hashes can be stored in a hash set, and any object not present in that set can be deleted.

Q6.2: Concurrent Garbage Collection Risk

Garbage collection is dangerous during a commit because the commit process may write new objects before updating the branch reference. During that gap, GC may think the new objects are unreachable and delete them. Git avoids this using locking, temporary object protection, and grace periods before deleting unreachable objects.

